

Synthetic Aperture Tilt-Shift from a Handheld Phone Video

ELEC 549 project report · Daniel Kuo · 22 September 2026

1. Creative goal

I wanted a single photograph of a real place whose plane of focus is *tilted through the scene*, taken with nothing but an iPhone. A tilt-shift lens can tilt its focus plane, which is what gives the miniature, model-village look; a phone lens cannot, and the tilt-shift filters in apps only paint a blur band across the picture. My goal was the real optical effect: every object blurred according to its true distance from a plane I choose after the fact.

The final image (Figure 1) looks down at an angle from an upper floor of O'Connor into the two-storey lobby. The focus plane lies along the upper floor, so the lounge furniture in the foreground is sharp, while the study table one floor below, the stairs and the wooden column right beside the camera fall away from that plane and blur by their own depth. Nothing in the image was blurred with a filter.



Figure 1. Final image: the O'Connor lobby from an upper floor. Focus plane tilted 26° off the lens axis, average of 169 aligned video frames spanning a synthetic aperture 0.8 of the sweep width.



Figure 2. The plain still from the middle of the same sweep (the reference frame). Everything is in focus, as a phone always renders it.

2. How it works

A lens with a wide aperture averages the views seen from every point across its opening. Objects on the focus plane line up in all of those views and stay sharp; objects off the plane land in different places and smear. A phone aperture is a few millimetres wide, so almost nothing smears. I instead moved the phone over a disc roughly an arm's reach across while recording video, so each frame is one view from one point of a very large aperture. Averaging the frames reproduces what a lens that size would have done (Levoy's SynthCam idea).

The alignment is what chooses the focus plane. Two views of any plane are related by a homography that depends only on the camera motion (R, t), the intrinsics K and the plane (normal n , distance d):

$$H = K (R + t n^T / d) K^{-1}$$

Nothing in this formula requires the plane to be visible or textured: pick any (n, d) , warp every frame by its H , average, and that plane comes out sharp. Tilting the plane is just changing n . A point at depth z blurs across a diameter of roughly $b = D f |1/z - 1/z_0|$ pixels, the circle-of-confusion formula with the sweep diameter D in place of the aperture diameter. Unlike SynthCam, which aligns on features the user taps, computing H from camera pose lets the plane pass through empty air or a featureless floor.

3. Capture

iPhone, 4K at 24 fps, SDR (BT.709), 1× lens. Each take is 42–50 s of video during which the phone is translated over a spiral while pointing the same way; small rotations are absorbed by the alignment. Every 5th frame is kept, giving 203–240 frames per take. Four scenes were shot on 18–20

September:

Take	Scene	Frames kept	Registered	Outcome
C003	Commons dining hall from the balcony	221	221 / 221	Two tilt directions rendered (Figs. 4–5)
C001	O'Connor lobby from an upper floor	213	213 / 213	Final image (Fig. 1)
C002	Quad from a roof at sunset, ~50 m away	240	240 / 240	Far scene; modest blur (Fig. 6)
C004	O'Connor lobby, second take	203	6 / 203	Pose failed; discarded

4. Technical steps

I wrote a small Python tool, `sa`, with one command per stage so that a bad pose run does not cost the extraction and a re-render costs neither. Every stage appends its parameters to `run.json`, so each output can be traced to the settings that made it.

Stage	Tool	What it does
ingest	ffmpeg	HEVC .mov → PNG frames, every Nth frame; records the transfer curve (sRGB or HLG)
pose	pycolmap (COLMAP on CPU)	SIFT features, sequential matching, incremental mapping → K, radial k, R and t per frame, sparse 3D cloud
plane	numpy	Click three points (fit a plane through the nearest sparse points), or click once to focus there and set tilt and azimuth
render	OpenCV	Undistort, convert to linear light, warp each frame by its H, accumulate in float64 with a weight mask, divide, re-encode
gui	FastAPI + WebGL	Browser page with a live low-resolution preview (~0.1 s per plane), and the same renderer ported to a GPU shader

Details that mattered

Reference frame and conventions. The output is rendered from the frame whose camera centre is nearest the centroid of all camera centres (the middle of the sweep), and the plane is expressed in that camera's coordinates. COLMAP poses are world-to-camera, so with the relative pose $R = R_i R_{\text{ref}}^T$, $t = t_i - R t_{\text{ref}}$ the homography carries a plus sign, $K(R + t n^T/d)K^{-1}$, where the camera-motion convention in my project plan had a minus. A synthetic test with known poses pinned the sign down before any real footage was shot.

COLMAP had to be re-tuned for a hand sweep. Its defaults expect photographs taken metres apart. A sweep of an arm's reach subtends only one or two degrees at a scene tens of metres away, below COLMAP's 16° minimum triangulation angle for the initial pair, so the mapper found no initial pair at all. Lowering the initial-pair gate to 2° and the triangulation and filtering angles to 0.2° made the reconstruction work. The focal length is weakly constrained in this regime, so it is seeded at $0.75 \times \text{image width}$ (about right for the 1× iPhone camera) rather than COLMAP's default 1.2.

Average in linear light. Video frames store gamma-encoded values. Averaging those directly renders the blurred regions too dark, so each frame is decoded to linear light through a lookup table, accumulated, and re-encoded once at the end (a unit test averages a black and a white frame and

expects 188, not 128). A weight image accumulated alongside keeps the borders, where not every frame contributes, from darkening.

Choosing the plane. Clicking the image sets the focus depth from the median depth of the nearest sparse points and makes that pixel the pivot of the tilt; tilt (degrees off fronto-parallel) and azimuth (direction of the tilt) are then set with sliders while a preview updates. Reconstructed points predicted to be within 2 px of focus are drawn in green, which makes the sharp band visible before rendering. An *aperture* control keeps only the frames whose camera centre lies within a fraction of the sweep radius, stopping the synthetic lens down.

Online demo. Because pose is solved once and rendering is cheap, the render stage could be ported to a WebGL fragment shader: each of the 60 frames of an exported scene is a texture layer, the shader applies that frame's H per pixel and accumulates in linear light, and a 960 px render takes about 3 ms on the GPU. The demo at <https://aperture-tilt-shift.vercel.app> renders the focus sweep live in the browser; the source is at <https://github.com/danielhkuo/aperture-tilt-shift>.

5. The final image in numbers

All quantities below come from `run.json` for take C001. Structure-from-motion has no absolute scale, so the reconstruction units are arbitrary; blur predictions in pixels only need the ratio of sweep diameter to depth.

Quantity	Value
Frames	213 kept from 1065 (every 5th); 213 registered by COLMAP
Intrinsics	$f = 2658$ px, radial $k = 0.015$, mean reprojection error 1.27 px
Sweep diameter D	10.4 units for the full sweep; aperture 0.8 keeps 169 frames, effective $D = 7.8$ units
Scene depth (5th–95th percentile of sparse points)	230–640 units, median 347
Focus plane	tilt 26.0° , azimuth 351° , depth 284 units at the pivot (the upper floor beside the sofa)
Reference frame	frame 0152, the camera nearest the centre of the sweep

Predicted blur $b = D f |1/z - 1/z_0|$ (pixel) at a few points of the scene, with z read from the sparse cloud and z_0 the depth of the tilted plane at that pixel:

Point	Depth z (units)	Predicted blur at 4K (px)
Upper-floor floor at the pivot (by the sofa)	284	0.0
Reception desk (left)	316	9.6
Top of the stairs	332	1.1
Table on the lower floor	626	33.2



Figure 3. Crops at full resolution, reference frame on the left and the render on the right. Top: the table one floor below (predicted ≈ 33.2 px) is smeared into a soft disc. Bottom: the sofa on the focus plane stays sharp, but the person who walked past during the sweep survives as a translucent ghost: a moving subject is averaged, not blurred.

6. Other results



Figure 4. Commons dining hall, plane tilted 38.5° so that it lies along the floor (azimuth 0° , aperture 0.75, 166 frames). Tables near and far are sharp; the banners hanging in front of the plane and the courtyard beyond it blur. Diners who moved during the sweep are ghosted.



Figure 5. Same video, same frames, plane tilted 63.5° in the opposite direction (azimuth 156°). The sharp band now runs from the upper left across the mezzanine while the floor below is out of focus. One capture, any plane.



Figure 6. Quad from a roof at sunset, plane tilted 81.5° , aperture 0.5 (133 frames). At roughly 50 m the parallax from an arm-span sweep is small and the blur is modest, as the formula predicts; the miniature effect needs nearer subjects or a bigger sweep.

7. Validation on a synthetic scene

Before trusting real footage I built a synthetic city (a textured ground plane seen from above with upright buildings), rendered a 60-frame spiral sweep from known poses, and ran the full pipeline on the encoded video. COLMAP recovered $f = 1001$ px against the true 1000 px and registered all 60 frames; clicking three ground points gave a plane tilted 54.8° against the true 55° . A second test renders Gaussian dots at known depths by direct point projection (sharing no code with the homography) and measures their rendered width: the measured blur matches $D f |1/z - 1/z_0|$ within 6 %, and a template-matched shift-and-add of the same frames agrees with the pose-derived warp to within 1 grey level, which checks the geometry two independent ways.



Figure 7. Tilt sweep of the synthetic scene from one capture, 0° to 60° in 10° steps, rendered in one pass over the frames. The sharp band moves from a fronto-parallel slab to the whole ground plane.

8. What I learned

The scene, not the code, is the main failure mode. Blur only appears where there is depth variation, and the amount scales with D/z . Looking down from height at two floors of a building (Figure 1) works; a distant quad (Figure 6) barely does. The predicted blur of ~ 33 px at 4K for the lower floor was visible but modest; a wider sweep would have made the miniature effect stronger.

Small-baseline structure-from-motion is fragile. Take C004 registered only 6 of 203 frames, and the first pose run on C001 also registered only 6 frames while a second run, with no change to the settings, registered all 213. COLMAP's initial-pair search is randomised and a sweep gives it very little triangulation angle to work with, so the fallbacks of re-running, or of aligning on tracked features for a fronto-parallel plane, remain necessary.

Focal length is poorly determined by a sweep. The same phone came out at 2658, 2697 and 2752 px across three takes, a 3.5 % spread. This hardly affects renders of a plane *picked* in the same reconstruction, because the reconstruction is self-consistent, but it biases numeric tilt angles.

Moving people ghost rather than blur (Figures 3 and 4). A mean over frames keeps a faint copy of anything that was present for part of the sweep. A median would drop them at the cost of changing the character of the blur; early-morning shoots avoid the problem altogether.

Stopping down trades blur for sharpness on the plane. The final renders all use the inner 50–90 % of the sweep rather than every frame. Pose error grows toward the ends of the sweep, where the camera is furthest from the reference view, and rolling shutter adds a per-frame skew, so the outermost frames cost more sharpness on the plane than they add in blur off it. Having the aperture as a live control made that trade-off visible.

Gamma matters in practice. Averaging gamma-encoded pixels would have darkened every blurred region; the transfer-curve material from class turned into a concrete design decision and a unit test.

The plane really can go where a lens cannot. Because the warp comes from pose rather than from image content, the plane in Figure 5 passes through the empty air of the atrium and through featureless walls, which is the case a feature-tracking approach cannot handle. And since pose is solved once, the render is cheap enough to run live in a browser: the whole family of images in Figures 4–5 and 7 comes from one 45-second clip each.